

CS323 Documentation

1. Problem Statement

Write a lexical analyzer for the language “Rat26S” either using a single finite state machine for the entire lexer, or using finite state machines for identifier, integer, and real.

Note: In your documentation (design section), YOU MUST write the REs for Identifiers, Real and Integer, and also show the NFSM using Thompson. Also the conversion to DFSM

The lexer

A major component of your assignment will be to write a procedure (Function) – `lexer()`, that returns a token when it is needed. Your `lexer()` should return a record, one field for the token and another field the actual "value" of the token (lexeme), i.e. the instance of a token.

Your main program should test the lexer i.e., your program should read a file containing the source code of Rat26S to generate tokens and write out the results to an output file. Make sure that you print both the tokens and lexemes.

2. How to use your program

This program was run and tested using the `g++.exe` compiler with version 13.2.0. In the same directory as the location of `lexical_analyzer.cpp`, please perform the following:

Step 1: How to compile (Windows): `g++ -o lexical_analyzer lexical_analyzer.cpp`

How to compile (Linux): `make`
(or using your desired compiler)

Step 2: How to execute:

There must be 2 command-line arguments given when performing the command for executing the program. These two arguments must be the names of text files. The first argument is an already existing input file which contains Rat26S source code. This is the file that will be read from. The second argument is the output file which will contain the generated tokens and lexemes. This file will be created once the program is finished. The format of the command is:

For Windows:

`./lexical_analyzer.exe "<input_file_name>.txt" "<output_file_name>.txt"`
ex) `./lexical_analyzer.exe "input_test_file_1.txt" "output_test_file_1.txt"`

For Linux:

`./lexical_analyzer "<input_file_name>.txt" "<output_file_name>.txt"`
ex) `./lexical_analyzer "input_test_file_1.txt" "output_test_file_1.txt"`

The created output .txt file should now appear in your current directory with the name you gave it, and it should contain the generated tokens and lexemes.

3. Design of your program

fsm.hpp

This is a C++ header file that contains functions which simulate deterministic finite state machines (DFSMs) for an identifier, integer, and real. Inside the file, the three functions are:

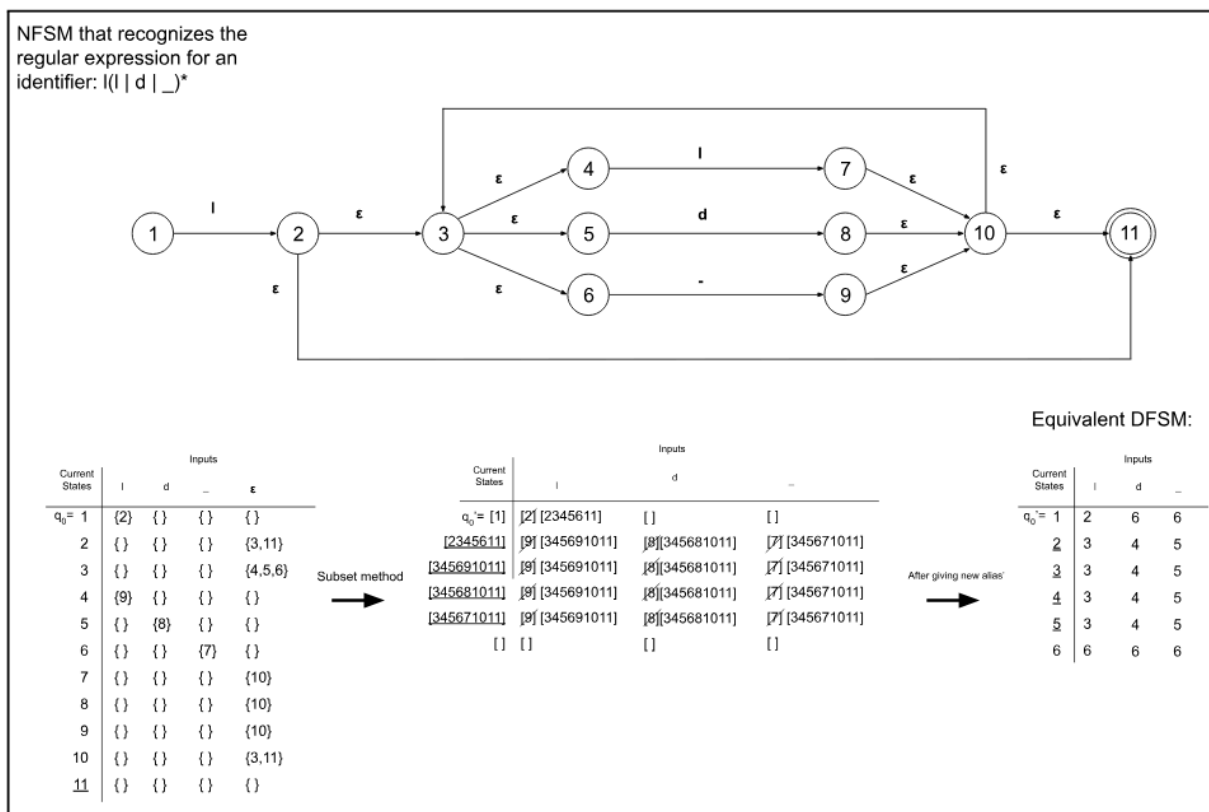
bool identifier_fsm(const std::string& w);

bool integer_fsm(const std::string& w);

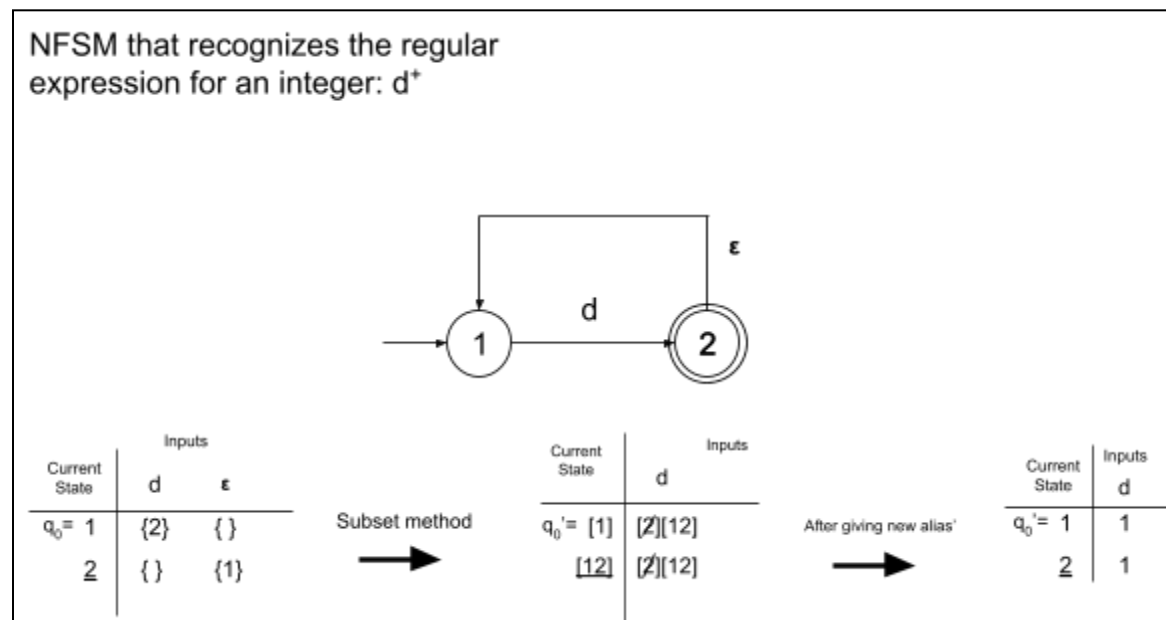
bool real_fsm(const std::string& w);

These are core functions which are called by the lexer in order to determine the token that a string is associated with. Each function takes an input string and returns a boolean indicating whether the string was accepted or rejected. Furthermore, each contains a next-state transition table which is represented using a 2-dimensional C++ vector container of integers. We obtained these state tables by using Thompson Construction to create non-deterministic finite state machines from regular expressions and then by using the Subset Method for creating the equivalent DFSM. The following images indicate the process of how we obtained each equivalent DFSM for a given regular expression.

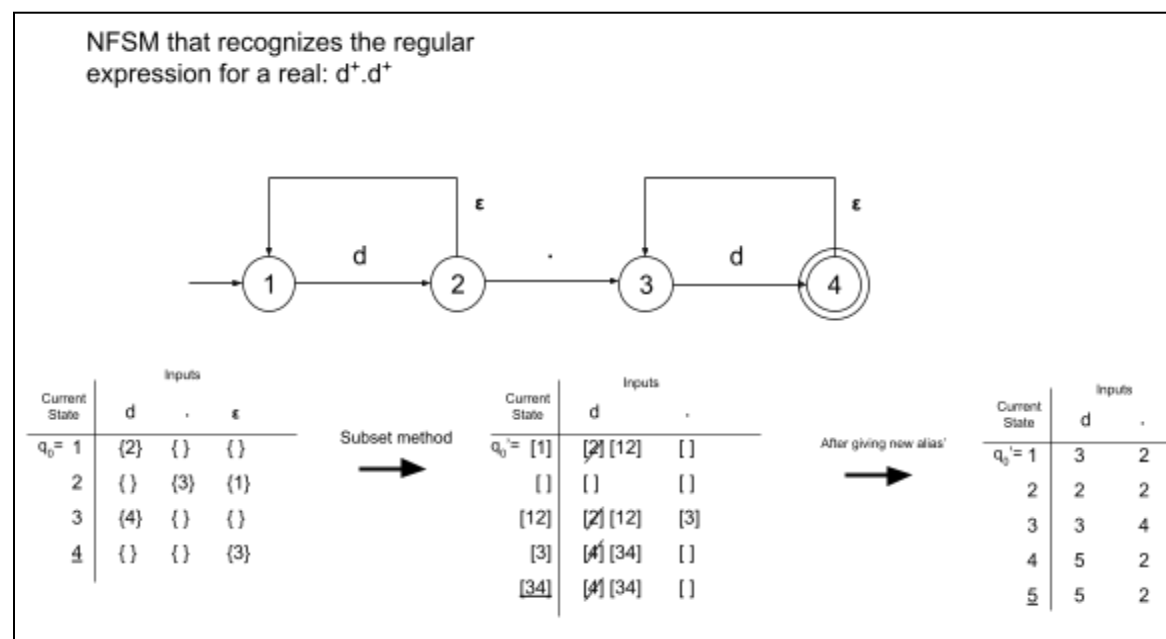
The process for obtaining the equivalent DFSM that recognizes an identifier:



The process for obtaining the equivalent DFSM that recognizes an integer:



The process for obtaining the equivalent DFSM that recognizes a real:



Record lexer(const std::string& input_string);

Our next core component is a function called `lexer` which takes a string as input and returns a `Record` object with two attributes: `token` and `lexer`. The attribute `token` corresponds with the token classification of the input string and the `lexeme` attribute corresponds with the instance of the token.

Inside the function, we have lists which hold our finite amount of separators, operators, and keywords. These lists were represented in code using C++ vector containers of strings. We compare the input string with the individual elements of these lists to see if there is a match. If none of these comparisons result in a match, then we call the previously mentioned DFSM functions and pass the input string as an argument for each call. If any one of the machines accepts the string, then we construct the appropriate `Record` object with the corresponding token `t` and return it.

The Main Loop

In the main entry point of the program, `main()`, we accept command line arguments where the first argument is the input text file containing Rat26S source code and the output file which will contain the generated tokens with their lexemes. We use C++ file streams to read and write from the input text file and the output text file.

Afterwards, we then implemented a while main loop which continuously reads the source code from the input file and writes the `Record` object data to the output file. To recognize tokens, our main loop reads a single line from the input file and identifies tokens by looking at the individual characters of the line of source code. We pass the character to the `lexer` function and view the obtained token. Depending on the token we obtained, we construct a lexeme and determine the final result.

For an identifier, we took keywords into account. If we find a character that was recognized as an identifier, then we see if there are any more letters that we need to add to the first character. We do this by continuously adding characters until the string no longer is recognized as an identifier and determine the end result. If the final constructed lexeme is considered an identifier and a keyword, then we prioritized the reserved keyword.

A similar process can be said about our system for recognizing operators. We first read a character and if it is recognized by the `lexer` function as an operator, we make sure to check for compound operators. If the single operator is the beginning character to a compound operator, then we read the next following character and check if it becomes a compound operator. If the

two characters make up a compound operator, we return the compound operator found. Otherwise, we return the original single operator found.

Our system for integers and reals is integrated together inside the main while loop. We must take into consideration that the integer portion of a real can be recognized as an integer before the entire real has been read. To counteract this, we first send the read character to the lexer which then identifies the character as an integer. Second, we construct the integer portion by continuously reading characters in the main loop. Third, we check and see if the next character is a decimal point and if so, we must keep reading. We then construct the fractional portion by reading characters after the decimal point which are recognized as an integer by the lexer function. Once the lexer function no longer recognizes the constructed fractional portion as an integer, we stop and add the three components together. With the original integer portion, the decimal point, and the potential fractional portion altogether, we check and see if they all create a real. If it is real, then we write the Record object data to the output file indicating that we found a real token. Otherwise, we write the Record data using the original integer we found in the first place.

The system for recognizing separators was much simpler in comparison to the others. We send the current character to the lexer function which then identifies the character as a separator token.

4. Any Limitation

'None'

5. Any shortcomings

'None'